

1) Implement the following using conditional operators

A) And gate B) OR gate C) Not gate D) Nand gate E) Nor gate F) Half Adder

```

1 // Code your testbench here
2 // or browse Examples
3 module tb;
4   reg a,b;
5   wire and_y,or_y,not_y,nand_y,nor_y,sum,carry;
6   cond_ops dut(a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
7   initial begin
8     a=0;b=0; #5 $display("a=%b b=%b and=%b or=%b not=%b
nand=%b nor=%b sum=%b carry=%b",
9
10    a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
11    a=0;b=1; #5 $display("a=%b b=%b and=%b or=%b not=%b
nand=%b nor=%b sum=%b carry=%b",
12
13    a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
14    a=1;b=0; #5 $display("a=%b b=%b and=%b or=%b not=%b
nand=%b nor=%b sum=%b carry=%b",
15
16    a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
17    $finish;
18  end
19 endmodule

```

```

1 // Code your design here
2 module cond_ops(input a, input b,
3   output and_y, or_y, not_y,
4   output nand_y, nor_y,
5   output sum, carry);
6   assign and_y = (a && b) ? 1 : 0;
7   assign or_y = (a || b) ? 1 : 0;
8   assign not_y = (a==1) ? 0 : 1;
9   assign nand_y = (a && b) ? 0 : 1;
10  assign nor_y = (a || b) ? 0 : 1;
11  assign sum = (a ^ b) ? 1 : 0;
12  assign carry = (a && b) ? 1 : 0;
13 endmodule
14
15

```

```

2 // or browse examples
3 module tb;
4   reg a,b;
5   wire and_y,or_y,not_y,nand_y,nor_y,sum,carry;
6   cond_ops dut(a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
7   initial begin
8     a=0;b=0; #5 $display("a=%b b=%b and=%b or=%b not=%b
nand=%b nor=%b sum=%b carry=%b",
9
10    a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
11    a=0;b=1; #5 $display("a=%b b=%b and=%b or=%b not=%b
nand=%b nor=%b sum=%b carry=%b",
12
13    a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
14    a=1;b=0; #5 $display("a=%b b=%b and=%b or=%b not=%b
nand=%b nor=%b sum=%b carry=%b",
15
16    a,b,and_y,or_y,not_y,nand_y,nor_y,sum,carry);
17    $finish;
18  end
19 endmodule

```

```

2 module cond_ops(input a, input b,
3   output and_y, or_y, not_y,
4   output nand_y, nor_y,
5   output sum, carry);
6   assign and_y = (a && b) ? 1 : 0;
7   assign or_y = (a || b) ? 1 : 0;
8   assign not_y = (a==1) ? 0 : 1;
9   assign nand_y = (a && b) ? 0 : 1;
10  assign nor_y = (a || b) ? 0 : 1;
11  assign sum = (a ^ b) ? 1 : 0;
12  assign carry = (a && b) ? 1 : 0;
13 endmodule
14
15

```

Log Share

```

# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: a=0 b=0 and=0 or=0 not=1 nand=1 nor=1 sum=0 carry=0
# KERNEL: a=0 b=1 and=0 or=1 not=1 nand=1 nor=0 sum=1 carry=0
# KERNEL: a=1 b=0 and=0 or=1 not=0 nand=1 nor=0 sum=1 carry=0
# KERNEL: a=1 b=1 and=1 or=1 not=0 nand=0 nor=0 sum=0 carry=1
# RUNTIME: Info: RUNTIME_0068 testbench.sv (16): $finish called.
# KERNEL: Time: 20 ns, Iteration: 0, Instance: /tb, Process: @INITIAL#7_0@.
# KERNEL: stopped at time: 20 ns
# VSIM: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.

```

Done

2) Implement the following using Bitwise operator and write Verilog code for the same

A=3'b101,B=3'b110,C=2'b10

- A&B=?
- ~(A|B)=?
- A^B=?
- ~(A^B)=?

on SystemVerilog Arrays
Sep 10, 2025
REGISTER NOW
Resources
Community
Help
Playgrounds
Pro

```

1 // Code your design here
2 // 2 - Bitwise operations
3 module bitwise_ops;
4     reg [2:0] A = 3'b101; // 5
5     reg [2:0] B = 3'b110; // 6
6     reg [1:0] C = 2'b10;
7
8     wire [2:0] A_and_B;
9     wire [2:0] not_AorB;
10    wire [2:0] A_xor_B;
11    wire [2:0] not_A_xor_B;
12
13    assign A_and_B      = A & B;          // A & B
14    assign not_AorB     = ~(A | B);       // ~(A|B)
15    assign A_xor_B      = A ^ B;          // A ^ B
16    assign not_A_xor_B  = ~(A ^ B);       // ~(A^B)
17
18    initial begin
19        $display("A = %b, B = %b", A, B);
20        $display("A & B      = %b", A_and_B);
21        $display("~(A | B)   = %b", not_AorB);
22        $display("A ^ B      = %b", A_xor_B);
23        $display("~(A ^ B)   = %b", not_A_xor_B);
24        $finish;
25    end
26 endmodule
27

```

1

```

1 // Code your design here
2 // 2 - Bitwise operations
3 module bitwise_ops;
4     reg [2:0] A = 3'b101; // 5
5     reg [2:0] B = 3'b110; // 6
6     reg [1:0] C = 2'b10;
7
8     wire [2:0] A_and_B;
9     wire [2:0] not_AorB;
10    wire [2:0] A_xor_B;
11    wire [2:0] not_A_xor_B;
12
13    assign A_and_B      = A & B;          // A & B
14    assign not_AorB     = ~(A | B);       // ~(A|B)
15    assign A_xor_B      = A ^ B;          // A ^ B
16    assign not_A_xor_B  = ~(A ^ B);       // ~(A^B)
17
18    initial begin
19        $display("A = %b, B = %b", A, B);
20        $display("A & B      = %b", A_and_B);
21        $display("~(A | B)   = %b", not_AorB);
22        $display("A ^ B      = %b", A_xor_B);
23        $display("~(A ^ B)   = %b", not_A_xor_B);
24        $finish;
25    end
26 endmodule
27

```

Log
Share

```

# KERNEL: ASUB file was created in location /home/runner/dataset.asap
# KERNEL: A = 101, B = 110
# KERNEL: A & B      = xxx
# KERNEL: ~(A | B)   = xxx
# KERNEL: A ^ B      = xxx
# KERNEL: ~(A ^ B)   = xxx
# RUNTIME: Info: RUNTIME_0068 design.sv (24): $finish called.
# KERNEL: Time: 0 ns, Iteration: 0, Instance: /bitwise_ops, Process: @INITIAL#18_1@.
# KERNEL: stopped at time: 0 ns
# VSIM: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Done

```

3) Write verilog code for Mux 16*1 using Gate level and Behavioral modelling.

WEBINARS

FREE • 1 Hour

Sep 10, 2025

NOW

ResourcesCommunityHelpPlaygroundsProfile

```

1 module tb;
2   reg [15:0] in;
3   reg [3:0] sel;
4   wire out;
5
6   mux16to1_behav dut(in, sel, out); // or mux16to1_gate
7
8   initial begin
9     in = 16'b1010_1100_1111_0001;
10    sel = 4'd0; #5 $display("sel=%d out=%b", sel, out);
11    sel = 4'd5; #5 $display("sel=%d out=%b", sel, out);
12    sel = 4'd10; #5 $display("sel=%d out=%b", sel, out);
13    sel = 4'd15; #5 $display("sel=%d out=%b", sel, out);
14    $finish;
15  end

```

```

2 //behaviour modelling
3 module mux16to1_behav(
4   input [15:0] in,
5   input [3:0] sel,
6   output reg out
7 );
8   always @(*) out = in[sel]; // procedural select
9 endmodule
10 //gate level
11 module mux16to1_gate(
12   input [15:0] in,
13   input [3:0] sel,
14   output out
15 );
16   assign out = in[sel]; // direct continuous assignment
17 endmodule

```

Log

Share

```

# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: sel= 0 out=1
# KERNEL: sel= 5 out=1
# KERNEL: sel=10 out=1
# KERNEL: sel=15 out=1
# RUNTIME: Info: RUNTIME_0068 testbench.sv (14): $finish called.
# KERNEL: Time: 20 ns, Iteration: 0, Instance: /tb, Process: @INITIAL#8_0@.
# KERNEL: stopped at time: 20 ns
# VSIM: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.

```

Done

4) Write Verilog code for ALU

New

Run

Save

Copy

KnowHow

WEBINARS

Everything on SystemVerilog Arrays

FREE • 1 Hour

Sep 10, 2025

REGISTER NOW

ResourcesCommunityHelpPlaygroundsProfile

```

1 module tb;
2   reg [3:0] a, b;
3   reg [1:0] sel;
4   wire [3:0] y;
5
6   alu dut(a,b,sel,y);
7
8   initial begin
9     a=4'd5; b=4'd3;
10
11    sel=2'b00; #5 $display("Add: %d", y); // 8
12    sel=2'b01; #5 $display("Sub: %d", y); // 2
13    sel=2'b10; #5 $display("AND: %b", y); // 0001
14    sel=2'b11; #5 $display("OR : %b", y); // 0111
15
16    $finish;
17  end

```

```

1 // Code your design here
2 module alu(
3   input [3:0] a, b,
4   input [1:0] sel, // 2-bit select
5   output reg [3:0] y
6 );
7   always @(*) begin
8     case(sel)
9       2'b00: y = a + b; // Add
10      2'b01: y = a - b; // Sub
11      2'b10: y = a & b; // AND
12      2'b11: y = a | b; // OR
13    endcase
14  end
15 endmodule

```

Log

Share

```

# KERNEL: Add: 8
# KERNEL: Sub: 2
# KERNEL: AND: 0001
# KERNEL: OR : 0111
# RUNTIME: Info: RUNTIME_0068 testbench.sv (16): $finish called.
# KERNEL: Time: 20 ns, Iteration: 0, Instance: /tb, Process: @INITIAL#8_0@.
# KERNEL: stopped at time: 20 ns
# VSIM: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.

```

Done

5) Write Verilog code for Decoder 4*1 using any levels of modelling.

```

1 module tb;
2   reg [1:0] s; wire [3:0] y;
3   decoder4to1 d(s,y);
4   initial begin
5     s=0; #5 $display("s=%b y=%b",s,y);
6     s=1; #5 $display("s=%b y=%b",s,y);
7     s=2; #5 $display("s=%b y=%b",s,y);
8     s=3; #5 $display("s=%b y=%b",s,y);
9   end
10 endmodule
11
1 // Code your design here
2 module decoder4to1(input [1:0] s, output reg [3:0] y);
3   always @(*) begin
4     y=4'b0000;
5     case(s)
6       2'b00: y[0]=1;
7       2'b01: y[1]=1;
8       2'b10: y[2]=1;
9       2'b11: y[3]=1;
10    endcase
11  end
12 endmodule
13
Log Share
KERNEL: Kernel process initialization done.
Allocation: Simulator allocated 4666 kB (elbread=427 elab2=4105 kernel=134 sdf=0)
KERNEL: ASDB file was created in location /home/runner/dataset.asdb
KERNEL: s=00 y=0001
KERNEL: s=01 y=0010
KERNEL: s=10 y=0100
KERNEL: s=11 y=1000
KERNEL: Simulation has finished. There are no more test vectors to simulate.
VSIM: Simulation has finished.
done

```

6) Write Verilog code for Mux 8*1 using different loops.

```

1 // Code your testbench here
2 // or browse Examples
3 module tb;
4   reg [7:0] d; reg [2:0] s; wire y;
5   mux8 m(d,s,y);
6   initial begin
7     d=8'b10101010;
8     for(s=0;s<8;s=s+1) begin #5 $display("s=%d y=%b",s,y);
9   end
10 end
11 endmodule
12
1 // Code your design here
2 module mux8(input [7:0] d, input [2:0] s, output reg y);
3   integer i;
4   always @(*) begin
5     y=0;
6     for(i=0;i<8;i=i+1)
7       if(s==i) y=d[i];
8   end
9 endmodule
10
Log Share
# Allocation: Simulator allocated 4665 kB (elbread=427 elab2=4104 kernel=134 sdf=0)
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: s=0 y=0
# KERNEL: s=1 y=1
# KERNEL: s=2 y=0
# KERNEL: s=3 y=1
# KERNEL: s=4 y=0
# KERNEL: s=5 y=1
# KERNEL: s=6 y=0
# KERNEL: s=7 y=1
# KERNEL: s=0 y=0
# KERNEL: s=1 y=1
# KERNEL: s=2 y=0

```

7) Write Verilog code for full adder 16-bit using full adder 4-bit using any one of the levels of modelling

WEBINARS

FREE • 1 Hour

Sep 10, 2025

NOW

3S

1 // Code your testbench here

2 // or browse Examples

3 module tb;

4 reg [15:0] a, b;

5 reg cin;

6 wire [15:0] sum;

7 wire cout;

8 fa16 uut(a, b, cin, sum, cout);

9 initial begin

10 a = 16'h00FF; b = 16'h0001; cin = 0;

11 #5 \$display("a=%h b=%h sum=%h cout=%b", a, b, sum,

12 cout);

13 a = 16'hFFFF; b = 16'h0001; cin = 0;

14 #5 \$display("a=%h b=%h sum=%h cout=%b", a, b, sum,

15 cout);

16 \$finish;

17 end

18 endmodule

1 // Code your design here

2 module fa16(

3 input [15:0] a, b,

4 input cin,

5 output [15:0] sum,

6 output cout

7);

8 assign {cout, sum} = a + b + cin; // Direct addition

9 endmodule

10

Log

Share

KERNEL: ASDB file was created in location /home/runner/dataset.asdb

KERNEL: a=00ff b=0001 sum=0100 cout=0

KERNEL: a=ffff b=0001 sum=0000 cout=1

RUNTIME: Info: RUNTIME_0068 testbench.sv (16): \$finish called.

KERNEL: Time: 10 ns, Iteration: 0, Instance: /tb, Process: @INITIAL#11_00.

KERNEL: stopped at time: 10 ns

VSIM: Simulation has finished. There are no more test vectors to simulate.

VSIM: Simulation has finished.

Done

8) Write Verilog code for decoder 4*1 using decoder 2*1 using any levels of modelling

WEBINARS

FREE • 1 Hour

Sep 10, 2025

NOW

1 // Code your testbench here

2 // or browse Examples

3 module tb;

4 reg [1:0] s; wire [3:0] y;

5 dec4 d(s,y);

6 initial begin

7 s=0; #5 \$display("s=%b y=%b",s,y);

8 s=1; #5 \$display("s=%b y=%b",s,y);

9 s=2; #5 \$display("s=%b y=%b",s,y);

10 s=3; #5 \$display("s=%b y=%b",s,y);

11 end

12 endmodule

3 assign y = (s==0)? 2'b01:2'b10;

4 endmodule

5

6 module dec4(input [1:0] s, output [3:0] y);

7 wire [1:0] y0,y1;

8 dec2 d0(s[0],y0);

9 dec2 d1(s[1],y1);

10 assign y = {y1[1]&y0[1], y1[1]&y0[0], y1[0]&y0[1],

11 y1[0]&y0[0]};

12 endmodule

Log

Share

KERNEL: Warning: You are using the Riviera-PRO EDU Edition. The performance of simulation is reduced.

KERNEL: Warning: Contact Aldec for available upgrade options - sales@aldec.com.

KERNEL: SLP simulation initialization done - time: 0.0 [s].

KERNEL: Kernel process initialization done.

Allocation: Simulator allocated 4667 kB (elbread=427 elab2=4105 kernel=134 sdf=0)

KERNEL: ASDB file was created in location /home/runner/dataset.asdb

KERNEL: s=00 y=0001

KERNEL: s=01 y=0010

KERNEL: s=10 y=0100

KERNEL: s=11 y=1000

KERNEL: Simulation has finished. There are no more test vectors to simulate.

VSIM: Simulation has finished.

Done

9) Write Verilog code for downcounter using loops

1

// Code your testbench here

2

// or browse Examples

3

module tb;

4

reg a,b; wire sum,carry;

5

halfadder h(a,b,sum,carry);

6

initial begin

7

a=0;b=0; #5 \$display("a=%b b=%b sum=%b

8

carry=%b",a,b,sum,carry);

9

a=0;b=1; #5 \$display("a=%b b=%b sum=%b

10

carry=%b",a,b,sum,carry);

11

a=1;b=0; #5 \$display("a=%b b=%b sum=%b

12

carry=%b",a,b,sum,carry);

13

a=1;b=1; #5 \$display("a=%b b=%b sum=%b

14

carry=%b",a,b,sum,carry);

15

end

16

endmodule

3

always @(*) fork

4

sum = a ^ b;

5

carry = a & b;

6

join

7

endmodule

Log

Share

```
# KERNEL: SLP simulation initialization done - time: 0.0 [s].
# KERNEL: Kernel process initialization done.
# Allocation: simulator allocated 4669 kB (e1bread=427 e1ab2=4108 kernel=134 sdf=0)
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: a=0 b=0 sum=0 carry=0
# KERNEL: a=0 b=1 sum=1 carry=0
# KERNEL: a=1 b=0 sum=1 carry=0
# KERNEL: a=1 b=1 sum=0 carry=1
# KERNEL: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
```

Done

REG NO:311123106055
ECE

